Automated Healthcare Diagnostic System using AutoML

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Mohammed Yaseen Agha (BU22CSEN0102188)
Nandish S K (BU22CSEN0102326)
P. Sujith (BU22CSEN0101493)
Nithin M (BU22CSEN0102394)

Under the esteemed guidance of
Dr. Arjun K P
Assistant Professor

GITAM
DEEMED TO BE UNIVERSITY

Department of Computer Science and Systems Engineering
GITAM School of CSE, GITAM (Deemed to be University)
Bengaluru
2026

DEPARTMENT OF COMPUTER SCIENCE AND SYSTEMS ENGINEERING

GITAM SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

GITAM

(Deemed to be University)

DECLARATION

We, hereby declare that the project report entitled **"Integrated Patient care dashboard for enhancing communication, medical record management, and service navigation within hospitals"** is an original work done in the **Department of Computer Science and Systems Engineering, GITAM School of Computer Science and Engineering, GITAM (Deemed to be University), Bengaluru** submitted in partial fulfilment of the requirements for the award of the degree of **B.Tech.** in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree.

Date:

Registration No(s)	Name(s)	Signature(s)
BU22CSEN0102188	**Mohammed Yaseen Agha**	
BU22CSEN0102326	**Nandish S K**	
BU22CSEN0101493	**P. Sujith**	
BU22CSEN0102394	**Nithin M**	

DEPARTMENT OF COMPUTER SCIENCE AND SYSTEMS ENGINEERING

GITAM SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

GITAM

(Deemed to be University)

CERTIFICATE

This is to certify that the project report entitled **"Automated Healthcare Diagnostic System using AutoML"** is a bonafide record of work carried out by Mohammed Yaseen Agha **(BU22CSEN0102188), Nandish S K (BU22CSEN0102326), P Sujit (BU22CSEN0101493), Nithin M (BU22CSEN0102394),** submitted in partial fulfillment of requirement for the award of degree of **Bachelors of Technology in Computer Science and Engineering**.

Project Guide	**Head of the Department**
SIGNATURE OF THE GUIDE	**SIGNATURE OF THE HoD**
Dr. Arjun K P	**Dr. Smita Sandeep Darandale**
Assistant Professor	Associate Professor and Head

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose consistent guidance and encouragement crowned our efforts with success

We consider it our privilege to express our gratitude to all those who guided us in the completion of the project.

We express our gratitude to Director **Prof. Y. Vamsidhar** for having provided us with the golden opportunity to undertake this project work in their esteemed organization.

We sincerely thank **Dr. Smita Sandeep Darandale,** HOD, Department of Computer Science and Systems Engineering, Gandhi Institute of Technology and Management, Bengaluru for the immense support given to us.

We express our gratitude to our project guide **Dr. Arjun K P,** Assistant professor, Department of Artificial Intelligence and Data Science, Gandhi Institute of Technology and Management, Bengaluru, for their support, guidance, and suggestions throughout the project work.

Student Name's	**Registration No.**
Mohammed Yaseen Agha	**BU22CSEN0102188**
Nandish S K	**BU22CSEN0102326**
P. Sujith	**BU22CSEN0101493**
Nithin M	**BU22CSEN0102394**

ABSTRACT

Modern healthcare systems generate large volumes of patient data, yet effective communication, centralized record management, and service navigation within hospitals remain fragmented and inefficient. Existing hospital information systems often operate in silos, leading to delays in accessing medical records, lack of real-time insights, and difficulty for patients in navigating hospital services. These limitations reduce operational efficiency and negatively impact patient experience and clinical decision-making. To address these challenges, this project proposes an Integrated Patient Care Dashboard that enhances communication, medical record management, and service navigation within hospitals through a unified digital platform.

The proposed system integrates a centralized dashboard that allows patients to securely access medical reports, personal documents, appointments, and healthcare insights, while enabling doctors to efficiently manage patient records and digital reports through a role-based interface. A hybrid data architecture is implemented using Supabase (PostgreSQL) for structured data and MongoDB for document storage, ensuring scalability and secure data handling. The system is developed using ReactJS for the frontend and FastAPI for backend services, with AI-assisted modules incorporating TF-IDF for keyword extraction, XGBoost for diabetes risk prediction, and LLM-based analysis for generating medical summaries and insights.

The results demonstrate improved accessibility of medical records, enhanced patient–doctor communication, intelligent report analysis, and simplified hospital navigation through integrated mapping features. The proposed solution provides a scalable, user-friendly, and AI-assisted healthcare management framework that supports efficient clinical workflows, better decision support, and an improved overall patient care experience within modern hospital environments.

TABLE OF CONTENTS

Title	Page No.
Declaration	**ii**
Acknowledgement	**Iv**
Abstract	**v**
Table of Contents	**vi**
List of Figures	vii
List of Tables	viii
1. INTRODUCTION	1-2
2. LITERATURE SURVEY	3-4
3. PROBLEM STATEMENT	5
3.1 Objectives	6
4. SOFTWARE AND HARDWARE SPECIFICATIONS	**7**
4.1 Introduction	7
4.2 Specific Requirements	7
4.2.1 Functional Requirement	7
4.2.2 Non-Functional Requirement	8
4.3 Hardware and Software Requirement	8
4.3.1 Hardware Requirement	8-9
4.3.2 Software Requirement	9-10
5. DESIGNING	**11**
5.1 System Architecture	11-12
5.2 Methodology	13-14
5.2.1 User based & Content based Features	14
5.2.2 Data Flow Diagram	15-16
5.2.3 Decision-Based Allocation Process	16-17
5.3 UML Diagrams	18-22
6. IMPLEMENTATION	**23-28**
7. TESTING	**30-31**
8. EXPERIMENTAL RESULTS	**32-34**
9. CONCLUSION	**35**
10. FUTURE WORK	**36-37**
REFERENCES	**38**
11. CO–PO–PSO MAPPING	**38-42**

LIST OF FIGURES

Fig No.	Title	Page
5.1	Application Hybrid Architecture of Automated Healthcare Diagnostic System	12
5.2.1	Use Case Diagram of Automated Healthcare Diagnostic System	14
5.2.2	Data Flow Diagram of Automated Healthcare Diagnostic System	16
5.4.1	Use Case Diagram of Automated Healthcare Diagnostic System	18
5.4.2	Sequence Diagram Showing CKD Prediction Flow	16
5.4.3	Class Diagram of Automated Healthcare Diagnostic System	20
6.2	Doctor Dashboard	24
6.3	Swagger APIs	25
6.5	AI Chat Feature	26
6.7	AI Generated Report Insights	27

1. INTRODUCTION

Modern healthcare systems generate vast amounts of patient data, including medical reports, diagnostic records, prescriptions, and appointment details. Effective management and accessibility of this information are essential for ensuring quality patient care and efficient hospital operations. However, many existing hospital systems are fragmented, with data stored across multiple platforms and departments, making it difficult for patients and doctors to access and utilize information efficiently. This lack of integration often leads to communication gaps, delayed decision-making, poor record management, and reduced overall healthcare efficiency.

Traditional hospital management systems primarily rely on isolated databases and manual coordination between departments. These systems often lack real-time data accessibility, intelligent insights, and user-friendly interfaces for patients and healthcare professionals. As a result, patients face difficulties in accessing their medical records, understanding complex reports, and navigating hospital services, while doctors struggle with scattered patient data and inefficient workflows. The absence of a centralized and intelligent dashboard reduces transparency, increases administrative burden, and limits effective communication within healthcare environments.

With the rapid adoption of digital healthcare technologies, there is a growing need for integrated, intelligent, and user-centric healthcare management solutions. Advanced web technologies, data analytics, and artificial intelligence offer significant potential in transforming how medical data is accessed, analyzed, and utilized. By providing a centralized platform that consolidates patient records, insights, and hospital services, healthcare institutions can improve operational efficiency, enhance patient engagement, and support better clinical decision-making.

This project proposes an Integrated Patient Care Dashboard designed to enhance communication, medical record management, and service navigation within hospitals through a unified digital interface. The system provides patients with a centralized dashboard to securely view medical reports, personal documents, appointments, and AI-generated insights, while enabling doctors to efficiently access patient data, upload reports, and manage clinical workflows through a role-based interface. Additionally, the system incorporates hospital navigation features that help patients locate departments, wards, and medical services with ease.

The application is developed using ReactJS for the frontend and FastAPI for backend services, with a hybrid database architecture utilizing Supabase (PostgreSQL) for structured data and MongoDB for document storage. The system also integrates intelligent modules such as TF-IDF for keyword extraction, XGBoost for diabetes risk prediction, and LLM-assisted analysis for generating medical

summaries and insights from patient records. This combination of modern technologies ensures scalability, secure data handling, and enhanced usability in real-world healthcare environments.

Despite significant advancements in healthcare digitization, hospitals still face challenges related to inefficient data sharing, limited patient engagement, and lack of intelligent decision-support systems. Patients often struggle to interpret medical reports and track their health progress, while healthcare providers encounter delays due to fragmented information systems and manual record handling. The absence of a comprehensive digital dashboard results in reduced transparency, increased workload for medical staff, and suboptimal patient experience.

Another major limitation of existing systems is the lack of real-time insights and personalized healthcare support. Most traditional platforms focus only on data storage rather than meaningful analysis and visualization. Furthermore, patients have limited visibility into their medical history, ongoing cases, and treatment progress, which can affect their understanding of health conditions and treatment plans. Similarly, hospitals face difficulties in providing seamless service navigation and coordinated care due to disconnected digital infrastructures.

The proposed system addresses these challenges by integrating centralized data management, AI-driven insights, and interactive dashboard visualization into a single platform. The use of intelligent analytics and automated summarization improves the interpretability of medical records, while role-based access control ensures secure and efficient data sharing between patients and doctors. The hybrid data architecture enhances performance and scalability, making the system suitable for real-world hospital environments.

By combining modern web development, artificial intelligence, and scalable database technologies, the Integrated Patient Care Dashboard provides a comprehensive solution for improving healthcare communication, record accessibility, and service navigation. The system aims to enhance patient experience, support informed clinical decisions, and contribute to the development of smarter and more efficient digital healthcare ecosystems.

2. LITERATURE SURVEY

The rapid advancement of digital technologies has significantly transformed the healthcare sector, particularly in the management, analysis, and sharing of medical data. Modern hospitals generate large volumes of patient information, including medical reports, diagnostic records, prescriptions, and treatment histories. Efficient management of this data is essential for improving patient care, clinical decision-making, and operational efficiency. However, many existing healthcare information systems remain fragmented, centralized, and dependent on manual workflows, leading to inefficiencies, poor interoperability, and limited accessibility of patient records. These challenges have encouraged researchers to explore integrated digital dashboards, artificial intelligence, and data-driven healthcare solutions to enhance medical record management and communication within healthcare environments.

Healthcare dashboards have emerged as an effective tool for visualizing and managing complex clinical data. Studies by Coiera et al. (2025) and Bischof et al. (2024) highlighted that digital clinical dashboards improve hospital workflow efficiency, patient monitoring, and decision support by providing real-time access to consolidated patient information. These systems enable healthcare professionals to quickly interpret patient data through visual analytics, reducing the cognitive burden associated with manual record review. Similarly, research on patient-centered dashboards emphasizes the importance of centralized platforms that allow patients to access medical records, reports, and healthcare insights in a structured and user-friendly manner.

Artificial intelligence has also gained significant attention in healthcare data analysis and medical record summarization. Keszthelyi et al. (2023) explored AI-based patient information summarization and found that automated analysis of clinical documents can significantly reduce the workload of healthcare professionals while improving the interpretability of medical reports. Generative AI and natural language processing techniques have been widely studied for extracting meaningful insights from unstructured medical data, enabling better communication between doctors and patients. TF-IDF and keyword extraction methods have been commonly used to identify important medical terms and enhance contextual understanding of healthcare documents.

Machine learning techniques have been increasingly applied in predictive healthcare analytics. Several studies have demonstrated the effectiveness of classification algorithms such as XGBoost in disease prediction and risk assessment due to their high accuracy and reliability in handling medical datasets. These models support early diagnosis and proactive healthcare management by analyzing historical patient data and generating predictive insights. The integration of machine learning with dashboard

systems allows healthcare providers to visualize predictive analytics alongside patient records, thereby improving clinical decision-making and personalized treatment planning.

In addition to AI and analytics, integrated hospital information systems have been widely researched to improve communication and coordination among healthcare stakeholders. Traditional hospital systems often operate in silos, where patient records, appointments, and reports are stored across multiple platforms, making data retrieval time-consuming and inefficient. Research on unified healthcare platforms suggests that centralized dashboards with role-based access control can significantly enhance collaboration between patients, doctors, and hospital staff while ensuring secure data access and management.

Recent studies have also explored the use of hybrid database architectures in healthcare applications to manage both structured and unstructured medical data efficiently. The combination of relational databases such as PostgreSQL and document-based databases like MongoDB has been recommended for scalable healthcare systems, as it enables efficient storage of medical records, reports, and analytics data. Cloud-based storage solutions and API-driven architectures further enhance system scalability, accessibility, and real-time data synchronization across healthcare modules.

Despite the significant progress in digital healthcare technologies, several challenges still exist. Data fragmentation, limited interoperability, lack of intelligent insights, and poor user experience remain major limitations of many existing healthcare management systems. Additionally, most traditional dashboards focus primarily on data storage and visualization rather than intelligent analysis, predictive modeling, and automated medical report interpretation. Security, privacy, and regulatory compliance also continue to be critical concerns in the design of healthcare information systems.

The proposed Integrated Patient Care Dashboard builds upon these research findings by implementing a full-stack, AI-assisted healthcare management system that integrates dashboard visualization, machine learning-based prediction, TF-IDF keyword extraction, and LLM-based medical insight generation. By combining modern web technologies, hybrid database architecture, and intelligent analytics, the system aims to enhance communication, improve medical record accessibility, support clinical decision-making, and provide a scalable and user-friendly solution for modern digital healthcare environments.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 PROBLEM STATEMENT

Modern hospitals generate and manage large volumes of patient data, including medical reports, prescriptions, diagnostic records, and appointment details. However, most existing healthcare systems are fragmented and operate through isolated platforms, making it difficult for patients and doctors to access and manage information efficiently. This lack of integration leads to delays in decision-making, poor communication, and inefficient clinical workflows within hospital environments.

Another major limitation of current systems is the absence of a centralized and intelligent interface for medical record management and analysis. Patients often struggle to understand complex medical reports and track their health information, while doctors must manually review scattered data across multiple systems. Additionally, many platforms focus only on data storage rather than providing meaningful insights, predictive analysis, and user-friendly visualizations that can support better healthcare decisions.

Furthermore, issues related to hospital service navigation, data security, and role-based accessibility continue to affect overall healthcare efficiency. Patients face difficulties in locating hospital services and accessing relevant information, while traditional centralized systems remain vulnerable to data breaches, unauthorized access, and single points of failure. These challenges highlight the need for an integrated, secure, and AI-assisted patient care dashboard that enhances communication, centralized record management, intelligent insights, and seamless navigation within modern hospital systems.

3.2 OBJECTIVES

The primary objective of this project is to design and develop an Integrated Patient Care Dashboard that enhances communication, medical record management, and service navigation within hospital environments. The system aims to provide a centralized and user-friendly platform where patients can securely access their medical reports, personal documents, appointments, and health-related information in a structured and efficient manner.

Another key objective is to enable healthcare professionals to efficiently manage patient data through a role-based interface that supports digital report uploads, record access, and streamlined clinical workflows. The project also focuses on integrating intelligent analytical features such as AI-based medical report summarization, TF-IDF keyword extraction, and machine learning-based risk prediction to generate meaningful insights and support better clinical decision-making.

Additionally, the system aims to improve patient experience and hospital efficiency by incorporating hospital navigation features, secure hybrid data storage, and scalable full-stack architecture. By integrating modern technologies such as ReactJS, FastAPI, Supabase, MongoDB, and AI modules, the proposed solution seeks to provide a secure, scalable, and intelligent healthcare platform that supports real-time data accessibility, enhanced usability, and future expansion for smart digital healthcare systems.

4. SOFTWARE AND HARDWARE SPECIFICATIONS

4.1 Introduction

The development of an efficient and intelligent Integrated Patient Care Dashboard requires well-defined software and hardware specifications to ensure proper functionality, performance, and scalability. This chapter describes the operational environment, system requirements, and technological components necessary for the design, development, and deployment of the proposed system. Since the platform handles sensitive medical records, patient data, and AI-generated insights, special emphasis is placed on data security, reliability, and system usability during the development process.

The proposed system follows a full-stack architecture where the frontend dashboard interacts with backend APIs, which in turn communicate with hybrid databases for structured and unstructured medical data management. The application integrates modern web technologies and AI modules to provide centralized access to medical records, intelligent report analysis, and hospital navigation features within a unified interface. This architecture ensures seamless data flow, efficient processing, and real-time accessibility for both patients and healthcare professionals.

The defined software and hardware specifications ensure that the system can operate smoothly in real-world hospital environments and support multiple users simultaneously. By utilizing scalable technologies such as ReactJS, FastAPI, Supabase, MongoDB, and AI-based analytical modules, the system is designed to deliver high performance, secure data handling, and reliable healthcare dashboard functionality suitable for modern digital healthcare ecosystems.

4.2 Specific Requirements

4.2.1 Functional Requirements

The functional requirements define the core services and operations that the Integrated Patient Care Dashboard must provide to its users, including patients and healthcare professionals. The system enables patients to create secure accounts and access their medical records, personal documents, appointments, and health insights through a centralized dashboard interface. All submitted and retrieved data is stored securely in the database architecture using Supabase for structured data and MongoDB for medical documents and reports.

The system also allows doctors to log in through a role-based interface to access assigned patient records, upload digital medical reports, manage case details, and monitor patient information efficiently. Backend APIs handle data validation, retrieval, and secure communication between the

frontend dashboard and the database, ensuring accurate and real-time data access. The platform further integrates AI-assisted modules to analyse medical reports, extract important keywords using TF-IDF, and generate readable summaries and insights for better understanding.

Additionally, the system includes intelligent features such as centralized report management, AI-based insight generation, diabetes risk prediction using machine learning models, and hospital service navigation through mapped department visualization. All critical operations such as authentication, data access, report uploads, and insight generation are executed through secure workflows with role-based access control, ensuring data privacy, system reliability, and efficient healthcare management within a unified digital platform.

4.2.2 Non-Functional Requirements

Non-functional requirements define the performance, security, reliability, and usability characteristics necessary for the effective operation of the Integrated Patient Care Dashboard. Security is a critical requirement due to the sensitive nature of healthcare data such as medical reports, personal records, and diagnostic information. The system ensures data protection through secure authentication mechanisms, encrypted communication, and controlled database access using role-based authorization. Sensitive data is stored securely using a hybrid database architecture with Supabase and MongoDB to maintain confidentiality and integrity.

Performance requirements ensure that the system provides fast response times for data retrieval, report viewing, and dashboard interactions. The application is designed to efficiently handle multiple user requests, real-time data processing, and AI-based insight generation without significant delays. Scalability is also considered so that the system can support an increasing number of patients, doctors, and medical records while maintaining consistent performance in a real-world hospital environment.

Reliability is maintained through proper error handling, stable API communication, and consistent database operations to prevent data loss or system failures. The system is designed with high availability and modular architecture to support continuous operation and future enhancements. Usability is another important aspect, and the dashboard interface is developed to be simple, intuitive, and user-friendly, enabling patients and healthcare professionals to easily access medical records, insights, and hospital navigation features without technical complexity.

4.3 Hardware and Software Requirements

4.3.1 Hardware Requirements

The proposed Integrated Patient Care Dashboard can be developed and executed using a standard computing environment with moderate hardware specifications. A system with at least an Intel i3 processor or equivalent is sufficient to support frontend development, backend services, database operations, and AI module integration. A minimum of 8 GB RAM is recommended to ensure smooth multitasking while running the React frontend, FastAPI backend, database services, and development tools simultaneously.

Adequate storage capacity is required to manage application files, medical documents, datasets, and project dependencies efficiently. A standard laptop or desktop with 256 GB or higher storage is suitable for development, testing, and deployment activities. Additionally, a stable and continuous internet connection is essential for accessing cloud databases (Supabase), API services, AI model integration, and real-time data synchronization between system modules.

The system is designed to be compatible with modern computing devices and web browsers, allowing development and testing on Windows, Linux, or macOS environments. Since the application is a web-based healthcare dashboard, it can operate efficiently on standard desktops and laptops used in hospital or clinical settings without the need for specialized hardware infrastructure.

4.3.2 Software Requirements

The software environment includes the tools, frameworks, and technologies required for frontend development, backend processing, database management, and AI integration. The system can be developed on operating systems such as Windows 10 or above, Linux distributions, or macOS. The frontend is implemented using HTML, CSS, and JavaScript with the ReactJS framework to create a dynamic, responsive, and user-friendly dashboard interface for patients and healthcare professionals.

The backend is developed using FastAPI, which handles server-side logic, API routing, authentication, and secure communication between the frontend and database. The system uses a hybrid database architecture where Supabase (PostgreSQL) is utilized for structured data such as user profiles, appointments, and authentication records, while MongoDB is used for storing unstructured data such as medical reports, documents, and AI-generated insights. This combination ensures efficient data management, scalability, and fast data retrieval.

For intelligent healthcare analysis, the system integrates AI and machine learning tools including Python libraries, TF-IDF (scikit-learn) for keyword extraction, and XGBoost for diabetes risk prediction. LLM APIs are used to generate medical summaries, insights, and explanations from patient reports, enhancing interpretability and decision support. Additional development tools such as Visual Studio Code are used for coding and debugging, while Docker is used for containerization and deployment in cloud environments.

The frontend and backend communicate through secure RESTful APIs, enabling real-time data submission, retrieval, and dashboard visualization. Proper validation mechanisms are implemented at both client-side and server-side levels to ensure accurate and complete data entry. Role-based authentication is incorporated to restrict access to sensitive medical data and ensure secure interaction between patients and doctors.

From a deployment perspective, the system is designed with a modular and scalable architecture that supports cloud deployment and future expansion. Cloud services and database hosting platforms can be integrated to support large-scale usage and real-time accessibility. The application is accessible through modern web browsers, eliminating the need for specialized client software and ensuring cross-platform compatibility.

These software design considerations ensure seamless data flow, secure medical record handling, efficient AI-assisted analysis, and reliable system performance, making the proposed dashboard suitable for real-world digital healthcare environments and future intelligent healthcare system integration.

5. DESIGNING

5.1 System Architecture

The design of the proposed Integrated Patient Care Dashboard focuses on providing a secure, scalable, and intelligent platform for managing medical records, patient communication, and hospital service navigation within a unified system. The system architecture follows a multi-layered structure, integrating a web-based dashboard, backend services, hybrid databases, and AI modules to support efficient healthcare data processing. This layered design enables seamless interaction between system components while ensuring data security, reliability, and real-time accessibility. The overall workflow and AI processing pipeline of the system are illustrated in Fig. 5.1.

As shown in Fig. 5.1, the system architecture consists of four main layers: the presentation layer, application layer, data storage layer, and AI analytics layer. The presentation layer is developed using modern web technologies such as ReactJS, providing an interactive dashboard interface for different users, including patients and doctors. Through this interface, users can securely log in, view medical records, upload medical reports, track appointments, access AI-generated insights, and navigate hospital services through an intuitive and user-friendly interface.

The application layer is implemented using FastAPI, which manages the core business logic of the system. This layer processes user requests, validates input data, manages authentication, and enforces role-based access control for both patients and healthcare professionals. It also handles API communication, medical report processing, AI insight generation, and interaction with the database services. Acting as a bridge between the frontend dashboard, AI modules, and storage systems, the application layer ensures smooth data flow and coordination between system components.

The data storage layer utilizes a hybrid database architecture to manage both structured and unstructured healthcare data efficiently. Supabase (PostgreSQL) is used for storing structured information such as user profiles, appointments, and authentication records, while MongoDB stores medical reports, documents, and AI-generated insights. This hybrid storage approach improves system scalability, performance, and efficient data retrieval while maintaining the secure handling of sensitive medical information.

The AI analytics layer, illustrated in Fig. 5.1, is responsible for intelligent data processing and insight generation. This layer integrates several AI and machine learning components, including TF-IDF for

keyword extraction, Sentence-BERT for semantic text representation, XGBoost for diabetes risk prediction, and LLM-based modules for medical report summarization and explanation. Medical reports uploaded by users are processed through text extraction and normalization, followed by feature extraction and machine learning analysis to predict health risks. The extracted information and predictions are then combined and processed by LLM modules to generate structured medical summaries and interactive explanations for users.

Overall, the layered architecture shown in Fig. 5.1 ensures modularity, scalability, and secure healthcare data management. By integrating modern web technologies, hybrid databases, and AI-driven analytics within a unified system, the proposed dashboard provides a reliable and efficient digital healthcare platform that enhances medical record accessibility, improves patient–doctor communication, and supports better hospital workflow management.

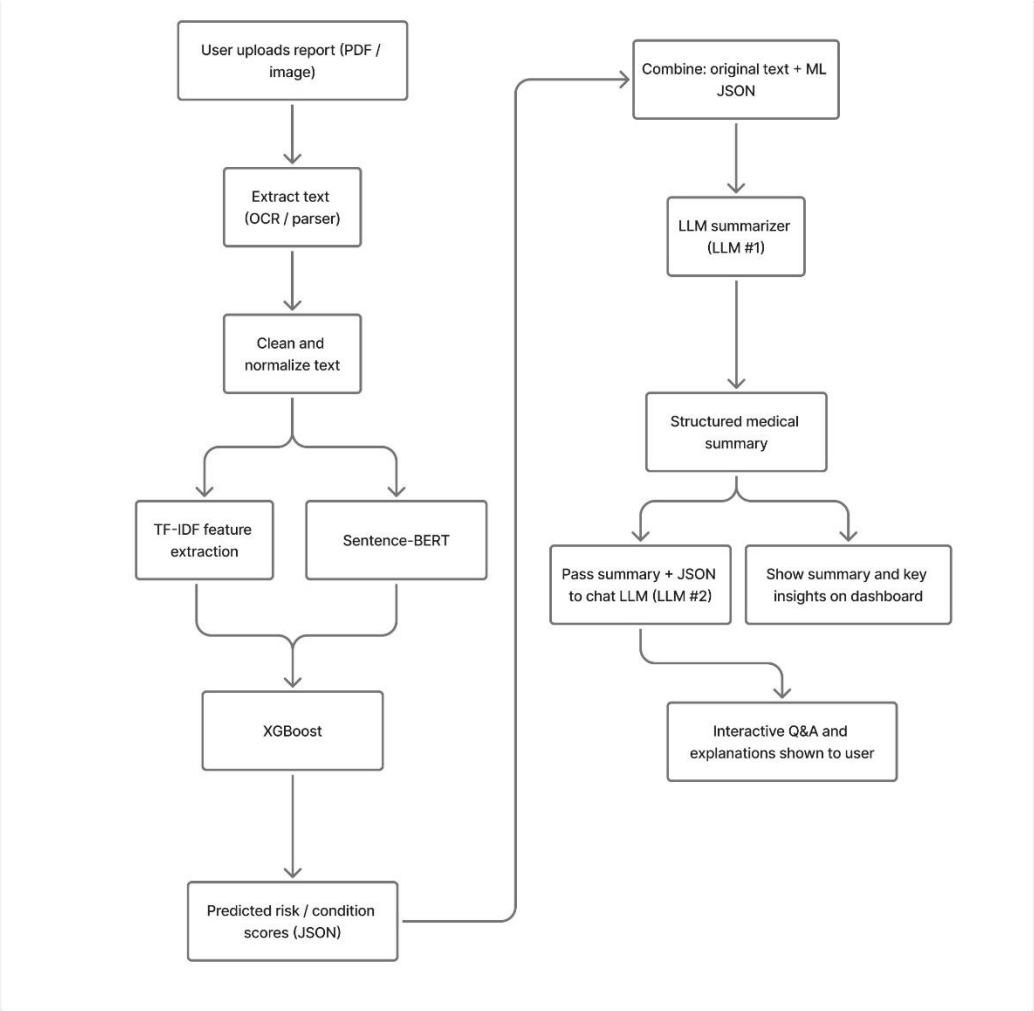

Fig 5.1 Application Hybrid Architecture of Automated Healthcare Diagnostic System

5.2 Methodology

The methodology adopted for the proposed system combines modern web application architecture, Fig 5.2, with artificial intelligence and hybrid database management to ensure efficiency, scalability, and secure healthcare data handling. The system follows a centralized yet modular approach where patient data, medical reports, and analytical insights are processed through an integrated dashboard platform. This approach enables efficient management of large volumes of medical data while ensuring accessibility, reliability, and real-time interaction between system modules.

When a user (patient or doctor) interacts with the system, the application first validates the submitted information through backend APIs and securely stores it in the hybrid database architecture. Structured data such as user profiles, appointments, and authentication details are stored in Supabase (PostgreSQL), while unstructured data such as medical reports, documents, and AI-generated insights are stored in MongoDB. This data management model ensures efficient storage, fast retrieval, and secure handling of sensitive healthcare information.

The methodology also incorporates AI-driven processing for intelligent healthcare analysis. Medical reports and user inputs are preprocessed and analyzed using TF-IDF for keyword extraction and machine learning models such as XGBoost for risk prediction. Additionally, LLM-based modules generate simplified summaries and explanations of medical reports to improve patient understanding and assist doctors in quick clinical interpretation. These analytical components enhance decision support and provide meaningful insights through the dashboard interface.

Furthermore, the system implements role-based access control to ensure that only authorized users can access and manage sensitive medical data. Patients can securely view their records and insights, while doctors can upload reports, access assigned patient data and monitor clinical information through a controlled interface. Secure API communication, input validation, and modular workflow design ensure system reliability, data privacy, and consistent performance in real-world healthcare environments.

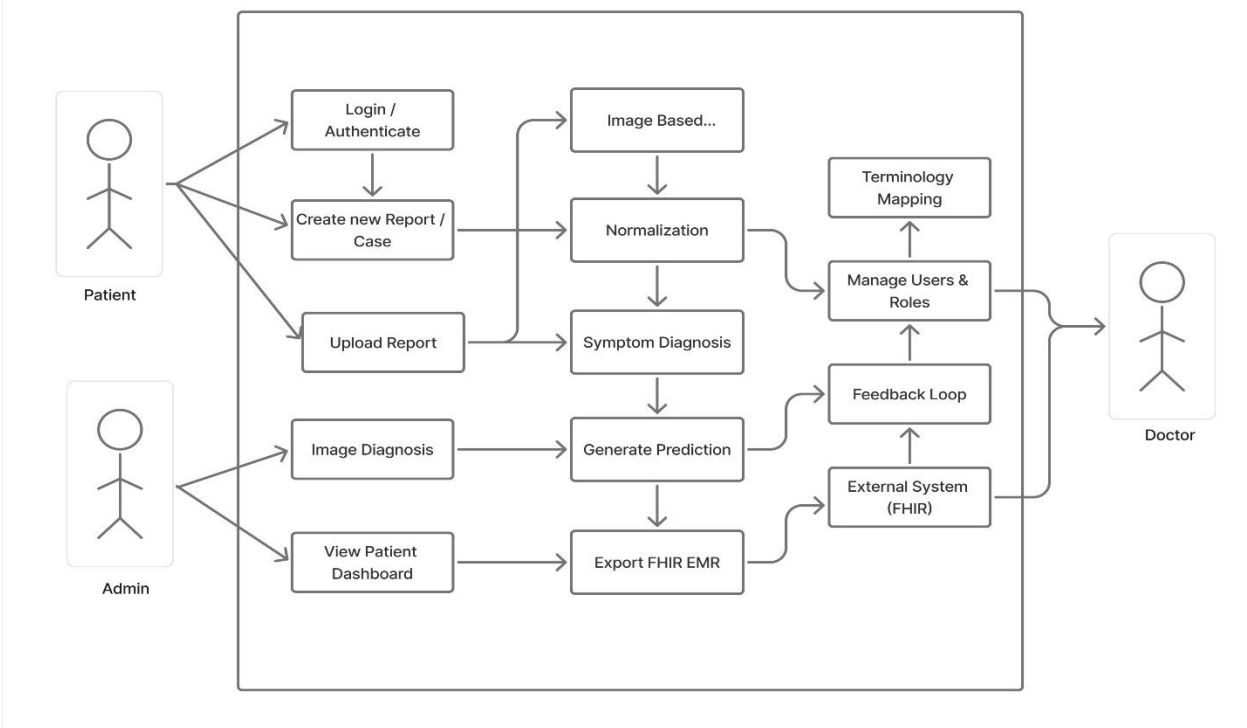

Fig 5.2.1 Use Case Diagram of Automated Healthcare Diagnostic System

5.2.1 User-Based and Content-Based Features

The system is designed to support multiple types of users, each with specific roles, permissions, and access levels to ensure secure and efficient healthcare data management. The primary users of the system include patients and doctors, along with administrative-level access for system management. Each user interacts with the dashboard based on their responsibilities and authorized privileges as shown in Fig. 5.2.1

Patients can register and securely log in to the system to access their medical reports, personal documents, appointment details, and AI-generated health insights through a centralized dashboard. They can upload medical records, view report summaries, track their cases, and navigate hospital services using integrated mapping features. The system ensures that patient data is stored securely and presented in a clear, user-friendly format to improve accessibility and understanding.

Doctors can access the system through a role-based interface that allows them to view assigned patient records, upload digital medical reports, analyse patient data, and monitor case progress efficiently. They can also utilize AI-assisted features such as medical report summarization and analytical insights to support faster clinical interpretation and decision-making.

Administrative control and role-based access mechanisms ensure that only authorized users can access sensitive medical information and perform critical operations such as report management and data updates. This structured user-based design enhances system security, protects patient privacy, and ensures controlled access to healthcare data while maintaining a seamless and intuitive user experience.

5.2.2 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how information moves through the proposed Integrated Healthcare Dashboard and how different system components interact to process and manage healthcare data. As shown in Fig. 5.2.2, the system facilitates secure communication between users, application modules, databases, and external healthcare systems.

The primary users of the system include patients, doctors, and administrative staff. These users access the platform through the dashboard interface after completing the user authentication process. Authentication ensures that only authorized users can access system functionalities based on their assigned roles and privileges. Once authenticated, users interact with the user dashboard, which acts as the central interface for managing medical records, viewing reports, and accessing healthcare services.

When a patient uploads medical reports or health-related documents, the data flows into the report management module, where the system processes and organizes the uploaded information. The processed data is then stored in the clinical database, which serves as the primary storage system for patient records, reports, and medical insights. This centralized database ensures secure storage and efficient retrieval of healthcare data.

The system also integrates an AI Assistant, which analyses the stored reports and medical data to generate intelligent insights and simplified explanations. These insights help both patients and doctors better understand medical reports and health conditions. The AI assistant retrieves relevant information from the database and presents meaningful summaries and guidance through the dashboard interface.

Additionally, the system communicates with external healthcare infrastructures through the Data Integration Layer, which enables interaction with hospital systems and FHIR-based healthcare platforms. This integration allows the dashboard to exchange medical data securely with hospital databases, improving interoperability and ensuring that patient records remain consistent across healthcare systems.

Overall, the data flow illustrated in Fig. 5.2.2 demonstrates how user inputs, AI modules, databases, and external healthcare systems interact within the Integrated Healthcare Dashboard. The structured flow of information ensures efficient data processing, secure storage, and reliable access to healthcare insights while supporting smooth communication between patients, healthcare professionals, and hospital systems.

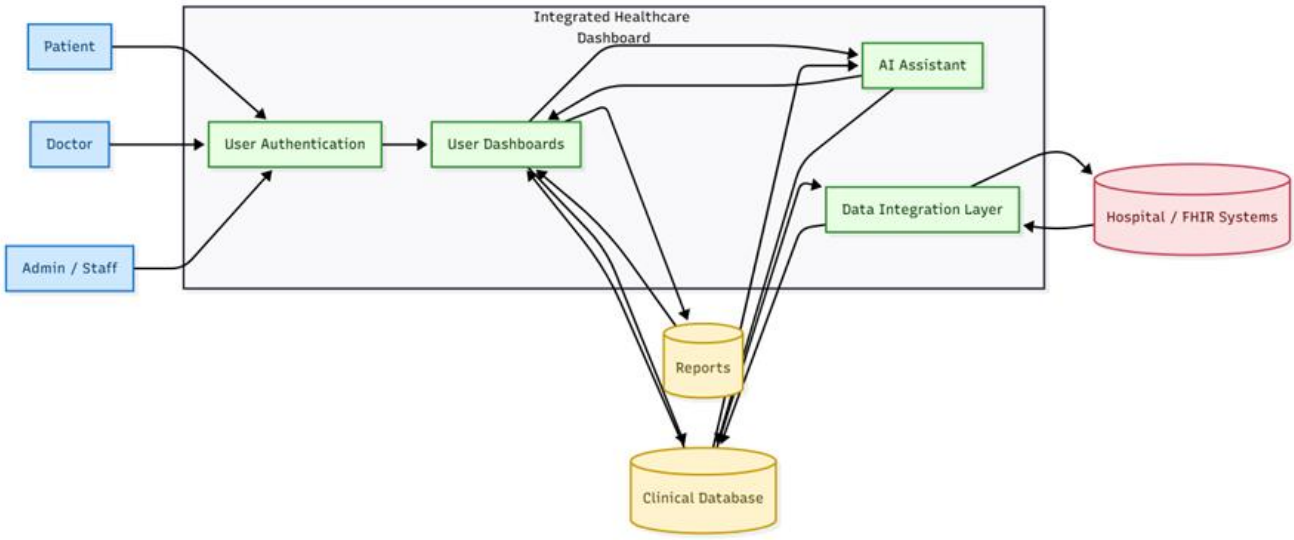

Fig 5.2.2 Data Flow Diagram of Automated Healthcare Diagnostic System

5.2.3 Decision-Based Allocation Process

The final decision-making process in the proposed system is based on validated medical data, AI-generated insights, and role-based verification rather than allocation mechanisms. The backend logic processes patient records, uploaded medical reports, and analytical outputs to generate meaningful summaries, risk predictions, and structured health insights. These decisions are governed by predefined validation rules, secure APIs, and AI-assisted analytical models to ensure accuracy, consistency, and reliability of healthcare information processing.

Once medical data or reports are uploaded, the system validates the input, stores it in the hybrid database, and initiates preprocessing and AI analysis. The generated insights, summaries, and risk predictions are then made available through the dashboard for both patients and doctors. Any updates to medical records, reports, or analytical results are securely stored and reflected in real-time within the system, ensuring transparency and consistent data availability for authorized users.

The overall system design emphasizes modularity and separation of concerns to ensure maintainability, scalability, and flexibility. Each functional component of the system, including user management, data processing, AI analysis, report management, and dashboard visualization, is implemented as an independent module. This modular architecture allows individual components to be upgraded or enhanced without affecting the entire system, thereby improving reliability and simplifying future integration with external healthcare platforms or hospital information systems.

The user interface is designed with a strong focus on simplicity, accessibility, and responsiveness to provide a seamless experience for patients and healthcare professionals. The dashboard follows a structured layout that includes medical report viewing, AI insights, appointment tracking, hospital navigation, and case management features. Client-side validation is implemented to ensure accurate data entry, while responsive design techniques enable the application to function effectively across different devices and screen sizes.

The backend design follows a layered architecture where request handling, business logic, and data access operations are organized into separate layers. This structure improves code readability, maintainability, and system performance. The backend is responsible for validating incoming requests, managing authentication, processing medical data, interacting with databases, and integrating AI modules for insight generation. Proper error handling, logging mechanisms, and secure API communication are incorporated to ensure stable system performance and fault tolerance.

A key aspect of the system design is the implementation of a hybrid data management model. Structured data such as user profiles, appointments, and authentication details are stored in Supabase (PostgreSQL), while unstructured data such as medical reports, documents, and AI-generated insights are stored in MongoDB. This design ensures efficient data retrieval, scalability, and secure storage of sensitive healthcare information while supporting real-time dashboard functionality.

From a system workflow perspective, the design follows a clear sequence of operations. When a patient or doctor interacts with the system, the data is first validated and securely stored in the database. The system then performs preprocessing and AI-based analysis to extract key medical insights and generate summaries. These results are displayed through the dashboard interface, enabling better understanding of medical reports and faster clinical interpretation.

The design also considers future scalability and interoperability. The modular and API-driven architecture allows easy integration with external healthcare systems, cloud infrastructure, and

advanced AI modules. The use of modern web technologies, hybrid databases, and intelligent analytics ensures that the system is adaptable for real-world hospital environments and capable of supporting large-scale digital healthcare applications in the future.

5.3 UML Diagrams

UML diagrams provide structural and behavioural representation of the system.

- **Use Case Diagram**

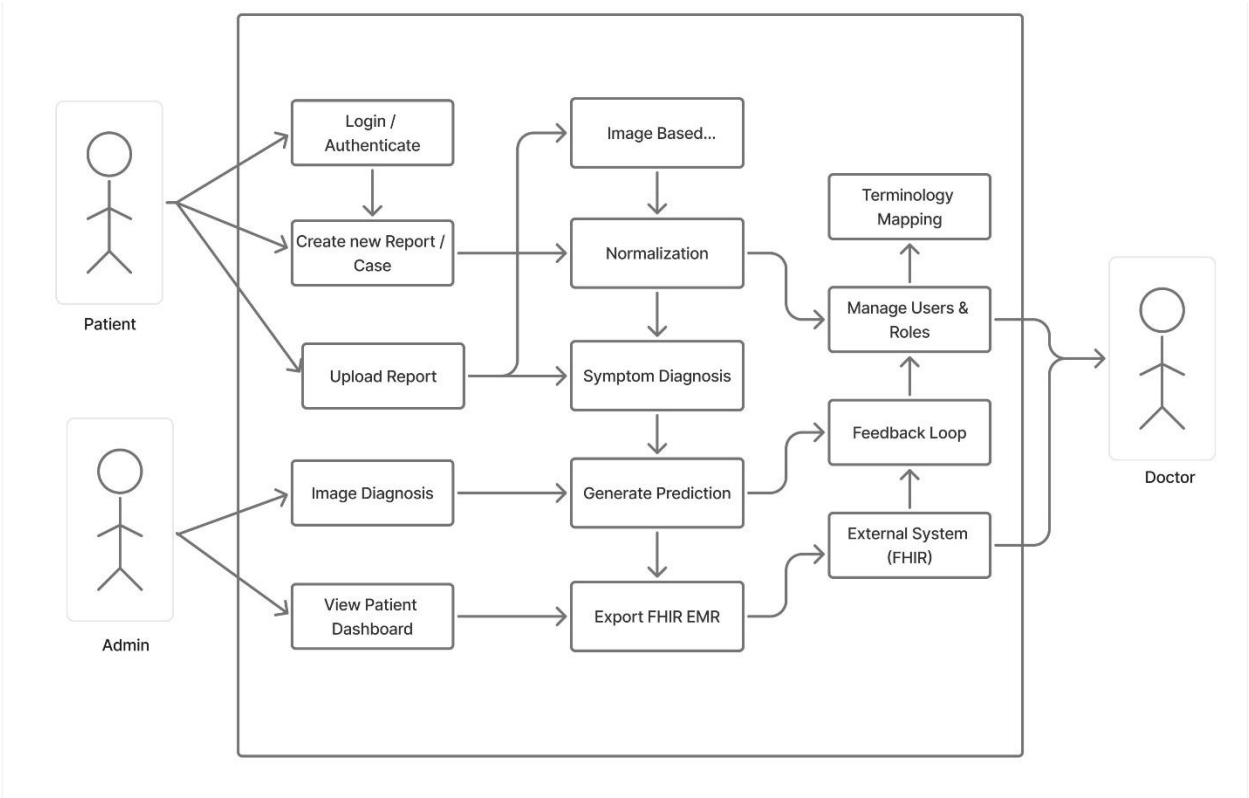

Fig 5.4.1 Use Case Diagram of Automated Healthcare Diagnostic System

Fig. 5.4.1 illustrates the use case diagram that represents the interactions between different system actors and the proposed Integrated Healthcare Dashboard. The diagram highlights how various users interact with the system to manage medical records, perform data analysis, and access healthcare services through the platform.

The primary actor, the patient, interacts with the system by logging in or authenticating through the dashboard. After authentication, the patient can create a new medical case, upload medical reports, and submit health-related information such as images or diagnostic data. The system then processes the uploaded reports through several analytical stages including image-based analysis, text

normalization, symptom diagnosis, and prediction generation, which help extract meaningful insights from the medical data.

The doctor acts as another important system actor who can review processed patient information and medical reports generated by the system. Doctors can interpret the diagnostic results, verify medical data, and use the insights generated by the platform to support clinical decision-making and patient consultation.

The admin or system administrator is responsible for monitoring and managing system operations. Administrative functions include managing users and roles, supervising system workflows, maintaining terminology mapping for medical data consistency, and monitoring system feedback mechanisms. The system also supports integration with external healthcare infrastructures through FHIR-based external systems, enabling secure data exchange with hospital systems and electronic medical record platforms.

- **Sequence Diagram**

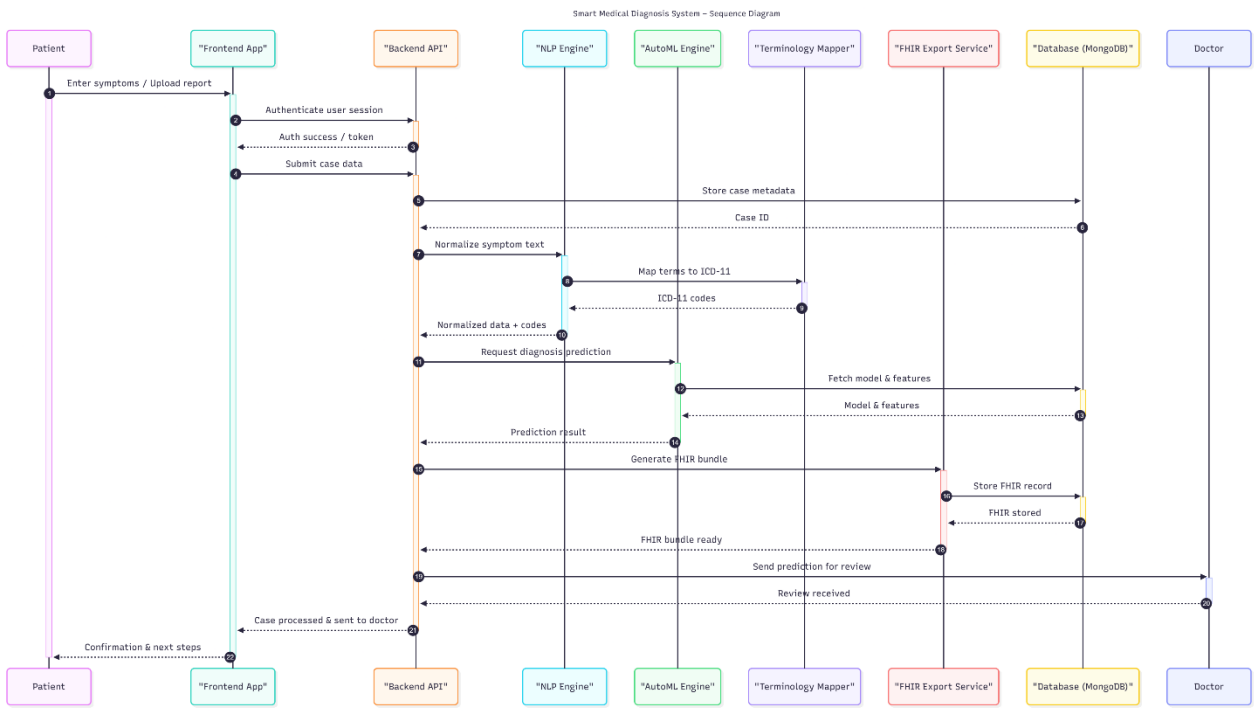

Fig 5.4.2 Sequence Diagram showing User and Approval Upload Flow

Fig. 5.4.2 illustrates the sequence diagram that describes the step-by-step interaction between the system components involved in processing patient medical data within the proposed Integrated Healthcare Dashboard. The diagram shows how different modules collaborate to analyze patient reports, generate predictions, and deliver insights to healthcare professionals.

The process begins when the patient enters symptoms or uploads a medical report through the frontend application. The frontend sends an authentication request to the backend API, which verifies the user session and returns an authorization token. After successful authentication, the patient submits the medical case data to the backend.

The backend API processes the request by storing case metadata and initiating text preprocessing. The NLP engine normalizes the symptom text and extracts relevant medical information. This processed information is then sent to the terminology mapping module, which converts extracted medical terms into standardized ICD-11 codes to ensure consistency with medical classification systems.

Once the medical data is normalized and mapped, the backend requests diagnostic analysis from the AutoML engine, which retrieves the trained model and relevant features to generate a diagnosis prediction. The prediction results are then returned to the backend system.

After the prediction is generated, the system creates a FHIR bundle through the FHIR export service, enabling interoperability with healthcare information systems. The generated medical record is then stored in the MongoDB database for secure data storage and future retrieval.

Finally, the diagnostic results and prediction summary are sent to the doctor for review, allowing healthcare professionals to examine the system-generated insights. Once reviewed, the system confirms that the case has been processed and communicates the status back to the frontend application, which notifies the patient and provides the next steps through the dashboard interface.

- **Class Diagram**

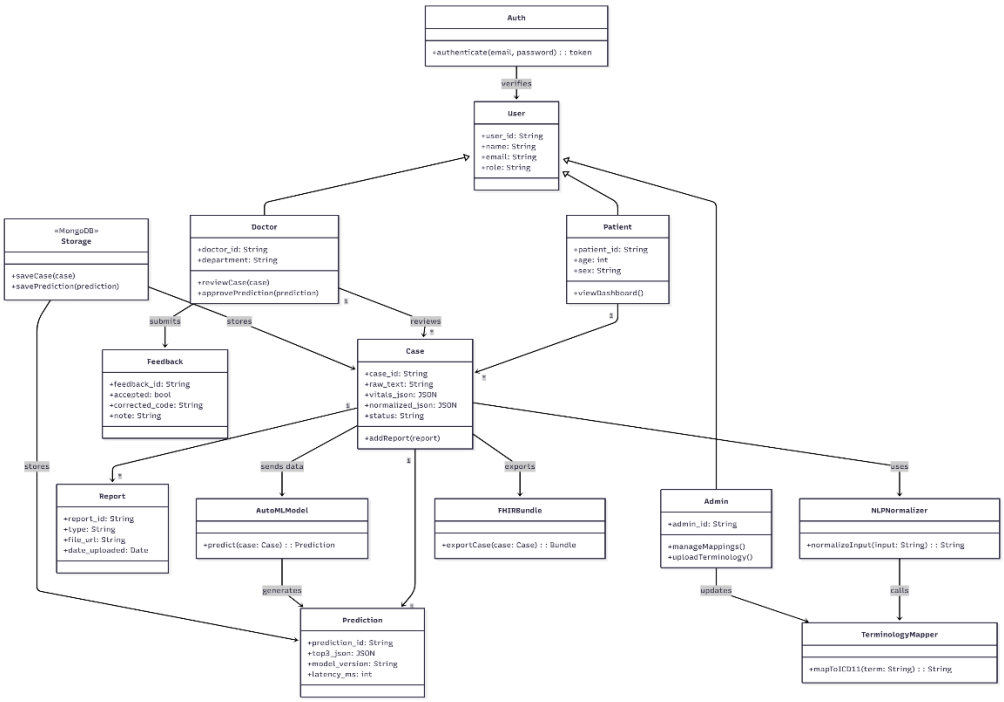

Fig 5.4.3 Class Diagram of Automated Healthcare Diagnostic System

The class diagram illustrates the structural design of the proposed Automated Healthcare Diagnostic System by defining the major system classes, their attributes, and the relationships between them, as shown in Fig. 5.4.3. This diagram provides a clear representation of how different components of the system interact to manage patient records, process medical reports, generate predictions, and integrate healthcare data.

The system includes the following primary classes:

- **Auth** – Responsible for user authentication and security verification. It manages login requests and generates authentication tokens using user credentials such as email and password.
- **User** – Represents the general user entity within the system and stores basic user information such as user ID, name, email, and role. Both patients and doctors inherit properties from this class.
- **Patient** – Represents patients using the healthcare dashboard. It stores patient-related information such as patient ID, age, and gender, and provides functions for accessing personal dashboards and medical records.
- **Doctor** – Represents healthcare professionals who interact with the system. Doctors can review submitted cases, analyze patient reports, and approve predictions generated by the AI modules.
- **Case** – Represents a medical case created when a patient uploads symptoms or reports. It stores case-related information such as case ID, raw symptom text, normalized medical data, and the current case status.
- **Report** – Stores uploaded medical reports and associated information including report ID, type, file path, and upload date.
- **Prediction** – Stores the results generated by the AI diagnostic model, including prediction ID, predicted conditions, model version, and processing latency.
- **AutoMLModel** – Represents the machine learning model responsible for analyzing patient cases and generating diagnostic predictions.
- **FHIRBundle** – Handles the generation of standardized **FHIR-based healthcare data bundles** that enable interoperability with hospital systems and electronic medical records.
- **NLPNormalizer** – Processes raw symptom text by performing normalization and preparing the data for further medical analysis.
- **TerminologyMapper** – Maps extracted medical terms to standardized medical codes such as **ICD-11**, ensuring consistent terminology across the healthcare system.

- **Admin** – Manages administrative operations including user management, terminology mapping updates, and system configuration.
- **Feedback** – Captures feedback or corrections provided by doctors during case review to improve system predictions and maintain data accuracy.
- **MongoDB Storage** – Represents the database layer responsible for securely storing cases, reports, predictions, and related healthcare data.

Relationships

- The Auth class verifies and authenticates the User before granting system access.
- The User class is associated with both Patient and Doctor, representing different user roles in the system.
- A Patient can create multiple Case records by uploading symptoms or medical reports.
- Each Case may contain one or more Report objects and is processed by the AutoMLModel to generate a Prediction.
- The NLPNormalizer processes case input text, and the TerminologyMapper converts extracted medical terms into standardized medical codes.
- The FHIRBundle class exports processed case data into standardized healthcare formats for integration with external hospital systems.
- Doctors review case predictions and may provide Feedback, which can be stored and used to improve future analysis.
- All system data, including reports, cases, and predictions, are stored in MongoDB Storage for secure data management.

6. IMPLEMENTATION

6.1 Introduction

The implementation phase focuses on the practical development of the Integrated Patient Care Dashboard based on the proposed system architecture and methodology. The system is developed as a full-stack web application that integrates frontend interfaces, backend APIs, hybrid databases, and AI analytical modules to provide centralized healthcare data management and intelligent medical insights. The implementation includes frontend dashboard development, backend service integration, database configuration, AI module integration, and system testing in a real-time web environment.

The application is designed to ensure secure medical record handling, role-based access, and seamless communication between patients and doctors while maintaining scalability, performance, and data privacy.

6.2 Frontend Implementation

The frontend of the system, Fig 6.2, is developed using HTML, CSS, JavaScript, and ReactJS to provide an interactive, responsive, and user-friendly dashboard interface. The frontend includes dedicated modules such as patient dashboard, doctor dashboard, medical reports, appointments, hospital navigation, and AI insights display.

React components are used to manage user interactions, state handling, and dynamic content rendering. The frontend communicates with the backend through secure RESTful APIs to fetch and display medical records, reports, and analytical insights in real time. Form validation mechanisms are implemented to ensure that users provide accurate and complete information during login, report uploads, and data entry.

The interface is designed with a clean layout and responsive design to ensure accessibility across different devices and screen sizes, enhancing overall user experience for both patients and healthcare professionals.

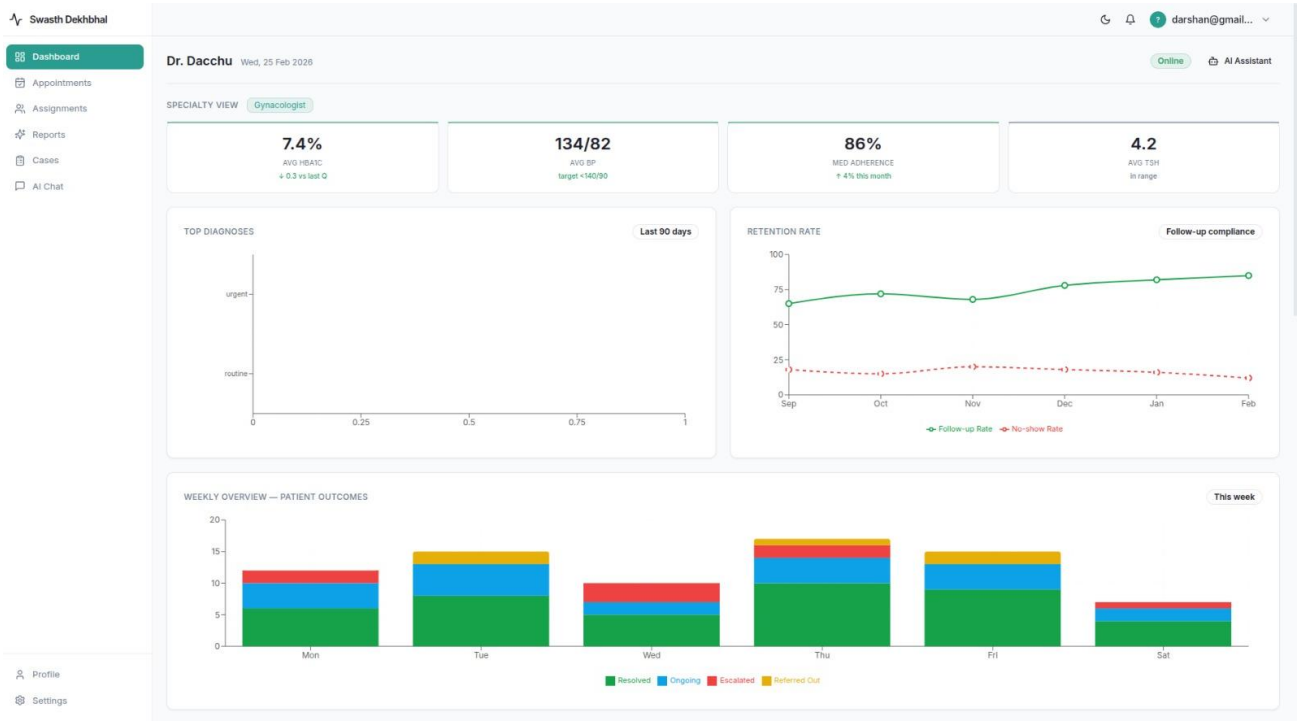

Fig 6.2. Doctor Dashboard

6.3 Backend Implementation

The backend of the system is developed using FastAPI, which handles business logic, API routing, authentication, and communication between the frontend, databases, and AI modules. RESTful APIs are created for user authentication, medical report management, data retrieval, AI analysis, and dashboard operations. Fig 6.3 shows the different APIs that have been developed for the system.

When a user uploads medical data or reports, the backend validates the input and securely stores it in the hybrid database. The backend also performs preprocessing of medical documents and integrates AI modules for keyword extraction, report summarization, and predictive analysis. Role-based access control is implemented to ensure that only authorized users such as doctors can access or modify sensitive patient information.

Additionally, the backend ensures secure data transmission, proper error handling, and efficient request processing to maintain system reliability and performance.

Fig 6.3. Swagger APIs

6.4 Database Implementation

The system uses a hybrid database architecture to efficiently manage structured and unstructured healthcare data. Supabase (PostgreSQL) is used for storing structured data such as user profiles, authentication details, appointments, and system metadata. MongoDB is used for storing unstructured data such as medical reports, documents, and AI-generated insights.

This hybrid storage approach enables fast data retrieval, scalability, and secure storage of sensitive medical information. Database schemas and collections are designed to organize patient records, doctor data, reports, and analytical results systematically. Secure database access and validation mechanisms are implemented to prevent unauthorized data access and ensure data integrity.

6.5 AI Module Implementation

The AI functionality is implemented using Python-based analytical libraries and external LLM APIs to provide intelligent healthcare insights. TF-IDF (Term Frequency–Inverse Document Frequency) is used for extracting important medical keywords from uploaded reports, while machine learning models such as XGBoost are used for diabetes risk prediction and health analysis.

Additionally, LLM-based modules are integrated to generate simplified medical summaries and explanations from complex medical documents. These AI components preprocess patient data, analyze

medical content, and generate readable insights that are displayed directly on the dashboard for better understanding and clinical interpretation. Fig 6.5 shows a patient asking the AI agent to cancel the an appointment. The system responds by cancelling it and showing other upcoming appointments.

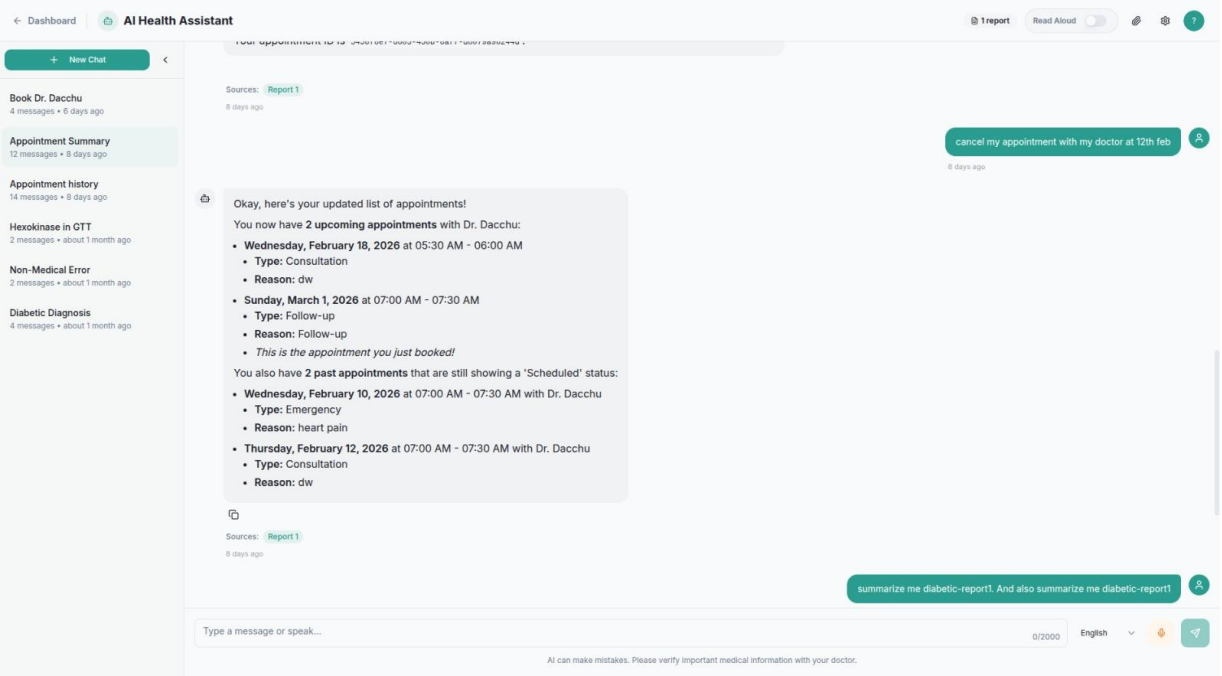

Fig 6. AI Chat Feature

6.6 System Integration

The system ensures seamless integration between frontend, backend, database, and AI components through RESTful API communication. JSON is used as the primary data exchange format to ensure lightweight and efficient data transmission. Secure API endpoints enable real-time interaction between modules such as report upload, insight generation, and dashboard visualization.

Environment configuration and dependency management are handled using environment variables to store sensitive credentials such as database URLs, API keys, and authentication secrets. This enhances system security and allows easy configuration across development, testing, and deployment environments.

6.7 Data Processing and Insight Generation

The data processing workflow begins when patients or doctors upload medical reports or enter health-related data into the system. The backend validates the data and stores it securely in the database. The

AI modules then preprocess the reports, extract key medical terms, and generate analytical summaries and risk predictions.

Doctors can review patient reports and insights through their dashboard, while patients can view simplified explanations and health summaries. This automated insight generation reduces manual effort in analyzing lengthy medical records and supports faster clinical decision-making. Fig 6.7 (a) and Fig 6.7 (b) shows the generated insight and other details that were extracted by the AI system for a particular report.

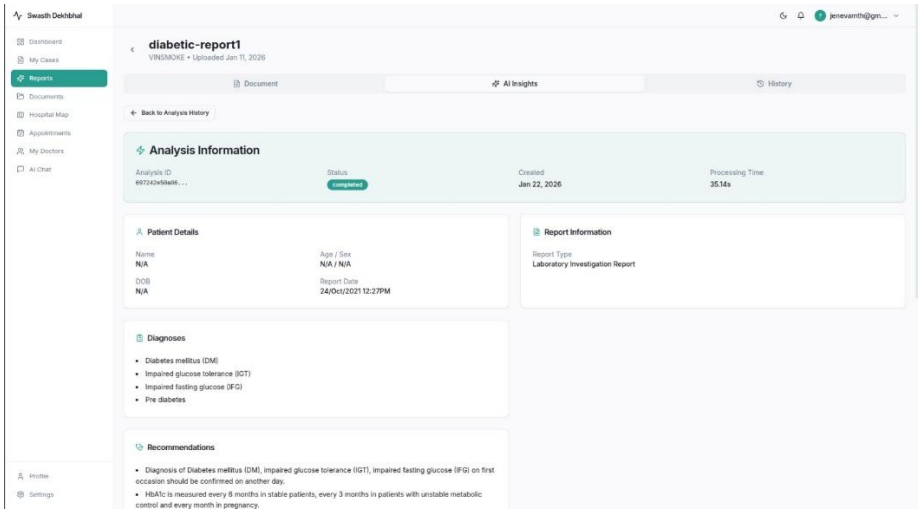

Fig 6.7 (a). AI Generated Report Insights

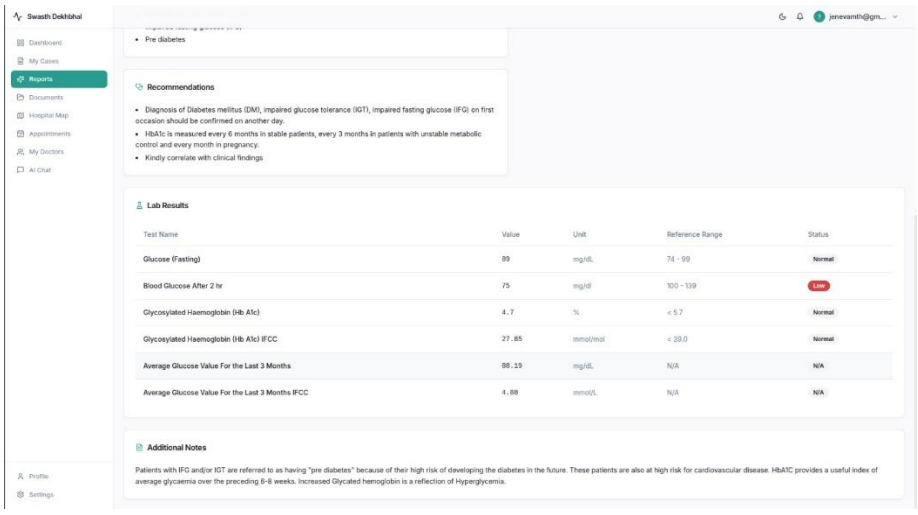

Fig 6.7 (b). AI Generated Report Insights

6.8 Testing Environment

The system is tested in a local development and cloud-compatible environment using modern development tools and web technologies. The application is executed using Visual Studio Code, with backend services running through FastAPI servers and databases configured using Supabase and MongoDB.

Functional testing is performed to validate modules such as authentication, report uploads, AI analysis, dashboard visualization, and role-based access. Integration testing ensures smooth communication between frontend, backend, and AI components. Proper input validation is implemented at both client-side and server-side levels to prevent incorrect or malicious data entry.

Logging mechanisms are incorporated to track important system activities such as user login, report uploads, AI processing, and API responses. Error handling strategies are also implemented to manage failures in database operations, API calls, or AI processing tasks.

Overall, the implementation successfully integrates full-stack web technologies, hybrid databases, and AI-driven analytics into a scalable healthcare dashboard system that ensures secure data handling, intelligent insights, efficient workflow management, and reliable performance in real-world healthcare environments.

7. TESTING

7.1 Introduction

Testing is a crucial phase in the software development lifecycle to ensure that the system functions accurately, securely, and efficiently under different operational conditions. The Integrated Patient Care Dashboard was tested to verify the proper functioning of all modules, including the frontend dashboard, backend APIs, database operations, AI analysis modules, and role-based authentication mechanisms. The testing process primarily focused on validating system functionality, data accuracy, performance, usability, and secure handling of sensitive medical information.

The application was tested in a local development environment using ReactJS for the frontend, FastAPI for backend services, and hybrid databases including Supabase and MongoDB. AI modules for medical report analysis and insight generation were also evaluated to ensure correct preprocessing, keyword extraction, and summary generation. The testing environment ensured smooth interaction between all system components in a real-time web application setup.

7.2 Types of Testing Performed

7.2.1 Unit Testing

Unit testing was conducted to verify the functionality of individual system components independently. Each module, such as user authentication, patient dashboard, doctor interface, medical report upload, AI insight generation, and navigation features, was tested separately to ensure correct input handling, processing, and expected output. This helped in identifying errors at an early stage and improving module-level reliability.

7.2.2 Integration Testing

Integration testing was performed to ensure seamless communication between different components of the system. The interaction between the React frontend, FastAPI backend, hybrid databases (Supabase and MongoDB), and AI modules was tested to confirm smooth data flow, accurate API responses, and real-time dashboard updates. This testing ensured that all integrated modules worked cohesively without data inconsistency or communication failures.

7.2.3 Functional Testing

Functional testing was carried out to verify that the system meets its intended objectives and user requirements. Various user scenarios were tested, including patient registration and login, secure access to medical records, report uploads, AI-generated insights, doctor access to patient data, and

hospital navigation features. The system was evaluated to ensure that all functionalities operated correctly and provided accurate outputs as designed.

7.2.4 AI Module Testing

AI module testing was conducted to validate the performance of analytical components such as TF-IDF keyword extraction, medical report summarization, and risk prediction models. The generated insights and summaries were analysed for accuracy, relevance, and clarity to ensure meaningful interpretation of medical data. This testing confirmed that the AI modules effectively supported clinical understanding and enhanced the usability of the dashboard system.

7.4 Performance and Security Testing

The system was tested for performance to ensure fast response times, smooth dashboard loading, and efficient processing of user requests. Since the application handles large medical records and AI-generated insights, performance testing focused on data retrieval speed, API response time, and real-time dashboard updates. The hybrid database architecture improved performance by storing structured data in Supabase and unstructured medical reports in MongoDB, enabling efficient data access and reduced latency during user interactions.

Security testing was conducted to ensure the safe handling of sensitive healthcare data, including medical reports, patient records, and analytical insights. The system implements secure authentication, encrypted API communication, and role-based access control to restrict unauthorized access. Only authorized users such as patients and doctors can access their respective dashboards and medical information, ensuring confidentiality and data privacy.

Additional security validation confirmed that protected modules such as report management, patient records, and AI insights could not be accessed without proper authentication. Input validation, secure session management, and controlled database access mechanisms were verified to prevent unauthorized data manipulation and ensure system reliability in real-world healthcare environments.

7.5 Results of Testing

All modules of the Integrated Patient Care Dashboard were tested successfully, and the expected outputs were achieved across all test cases. The system demonstrated reliable functionality in user authentication, medical report uploads, AI insight generation, dashboard visualization, and role-based

data access. The testing results confirmed proper integration between the frontend, backend APIs, databases, and AI modules.

In addition to functional validation, the system was tested under different operational scenarios to ensure stability and consistent performance. Multiple test iterations were conducted to evaluate system behavior during repeated user interactions such as patient login, report uploads, and data retrieval. The results confirmed that the system-maintained data consistency across the hybrid database and provided accurate real-time dashboard updates without errors or data loss.

User interface testing was performed to verify that all forms, navigation elements, and dashboard components functioned correctly. Each input field was tested with both valid and invalid data to ensure proper client-side and server-side validation. Appropriate error messages were displayed for incomplete or incorrect inputs, improving usability and preventing invalid data submission.

Backend API testing was conducted to ensure proper request handling, response generation, and database operations. Each API endpoint was tested using multiple input scenarios to validate successful data storage, retrieval, and AI processing. The system was also evaluated for handling invalid requests, network interruptions, and error conditions. Logging mechanisms were used to monitor system activity and identify potential issues during testing.

Security testing confirmed that unauthorized users could not access restricted modules or sensitive medical data. Attempts to access protected dashboards and records without proper authentication were successfully blocked by the role-based access control system. This ensured that patient privacy and data confidentiality were maintained throughout system operation.

Performance testing indicated that database operations, dashboard rendering, and API responses were executed efficiently with minimal delay. Stress testing with multiple concurrent user interactions showed that the system remained stable without crashes or performance degradation. The modular architecture and optimized data handling contributed to smooth system performance even under repeated usage conditions.

8. EXPERIMENTAL RESULTS

8.1 Introduction

The experimental evaluation of the proposed Integrated Patient Care Dashboard was conducted to analyze its functionality, performance, accuracy, and reliability in a real-time web application environment. The experiments focused on validating key system modules such as user authentication, medical report management, AI-based analysis, dashboard visualization, and role-based data access. The goal of the evaluation was to ensure that the system operates efficiently while providing secure handling of healthcare data and meaningful analytical insights.

The application was deployed in a local development environment using ReactJS for the frontend, FastAPI for backend services, and a hybrid database architecture consisting of Supabase (PostgreSQL) and MongoDB. The AI modules for report summarization, keyword extraction, and risk prediction were also tested using real and sample medical datasets to evaluate analytical performance and usability.

8.2 Patient Registration and Dashboard Results

The first experiment involved registering multiple users (patients and doctors) through the web interface and accessing the dashboard functionalities. After submission, the system successfully validated user inputs and stored the information securely in the database. The registered users were able to log in and access their respective dashboards without errors.

The experiment confirmed that the system provides secure authentication, accurate data storage, and seamless dashboard access. Patient records, appointments, and uploaded reports were retrieved in real time, demonstrating efficient database integration and smooth frontend-backend communication.

The experiment confirmed that the system can securely register users and maintain tamper-proof records.

8.3 Medical Report Analysis Results

To evaluate the analytical capabilities of the system, multiple medical reports were uploaded and processed through the AI modules. The system successfully performed preprocessing, keyword extraction using TF-IDF, and generated simplified summaries using LLM-based analysis. The extracted keywords and generated insights were displayed clearly on the dashboard for easy interpretation.

The experiment confirmed that the system can effectively analyze complex medical documents and present meaningful insights that improve patient understanding and assist doctors in faster clinical evaluation.

8.4 Data Processing and Insight Generation

The data processing workflow was tested by submitting various health-related inputs and medical records into the system. The backend successfully validated and stored the data in the hybrid database and initiated AI-based analysis. Machine learning models such as XGBoost were used for risk prediction, and the results were displayed through the dashboard interface.

The experimental results showed that the system generates accurate insights, structured summaries, and predictive outputs without delays. This demonstrates the effectiveness of integrating AI analytics within a centralized healthcare dashboard.

8.5 System Performance Analysis

The performance of the system was evaluated based on dashboard loading time, API response speed, and database query efficiency. The average response time for data retrieval and dashboard rendering was observed to be fast and stable under normal usage conditions. Hybrid storage using Supabase and MongoDB improved data access speed and reduced latency during report viewing and insight generation.

Stress testing with multiple user interactions confirmed that the system handled concurrent requests efficiently without crashes or data inconsistency. Backend APIs processed requests smoothly, and the modular architecture ensured stable performance during repeated operations.

8.6 Security and Access Control Evaluation

The system was evaluated for security by performing multiple unauthorized access attempts and role-based permission tests. Users without proper authentication were restricted from accessing sensitive modules such as medical records, doctor dashboards, and analytical insights. Role-based access control mechanisms successfully ensured that patients and doctors could only access their authorized data.

Additional validation confirmed that secure API communication, input validation, and database access controls prevented unauthorized data manipulation. The system-maintained confidentiality and integrity of sensitive healthcare data throughout all testing scenarios.

8.7 Summary of Experimental Results

The experimental evaluation demonstrated that the proposed system performs efficiently, securely, and reliably in a real-time healthcare environment. The major observations include:

• Successful user registration and secure authentication

• Efficient storage and retrieval of medical records using hybrid databases

• Accurate AI-based medical report summarization and keyword extraction

• Reliable dashboard performance with real-time data updates

• Effective role-based access control and secure data handling

• Stable system performance under multiple user interactions

Further experiments were conducted to evaluate system stability under repeated usage conditions. Multiple patient records and report uploads were processed simultaneously to observe system responsiveness and consistency. The backend handled concurrent requests without data loss or performance degradation, confirming the scalability of the architecture.

User interface performance was also analyzed to ensure smooth interaction with dashboard components, forms, and navigation modules. The application maintained consistent responsiveness across different test scenarios and device environments.

Overall, the experimental results confirm that the Integrated Patient Care Dashboard successfully meets its design objectives by providing centralized medical record management, AI-assisted analysis, secure data access, and efficient system performance. The system demonstrates strong feasibility for real-world deployment in digital healthcare environments, enhancing patient engagement, clinical decision support, and overall healthcare management efficiency.

9. CONCLUSION

Healthcare systems today struggle with fragmented data, poor interoperability, and limited real-time support, slowing down clinical decisions and reducing efficiency. This project addresses these issues through an Integrated Patient Care Dashboard that combines modern web technologies, a hybrid database architecture, and AI-driven analytics into a single, centralized platform. The system uses Supabase for structured data and MongoDB for unstructured medical documents, ensuring scalable and secure storage. AI features including medical report summarization, TF-IDF keyword extraction, and ML-based risk prediction help both patients and doctors interpret health data more effectively.Role-based access control ensures that patients can securely view their own records while doctors manage clinical information through a controlled interface. Authentication, input validation, and secure APIs further protect data privacy.Testing confirmed stable performance across all modules; authentication, dashboard visualization, report management, and AI analysis, with fast data retrieval and accurate outputs. The modular design also supports future enhancements such as mobile access, cloud deployment, and integration with hospital systems. In summary, the Integrated Patient Care Dashboard delivers a reliable, intelligent, and scalable solution for modern healthcare data management, improving accessibility, clinical decision-making, and coordination between patients and healthcare professionals.

10. FUTURE WORK

The proposed Integrated Patient Care Dashboard successfully demonstrates a centralized and intelligent platform for managing medical records, AI-based insights, and patient–doctor communication. Although the system achieves its core objectives of secure data management, dashboard visualization, and AI-assisted analysis, several enhancements can be incorporated in future versions to improve scalability, functionality, and real-world applicability in advanced healthcare environments.

One major area of future improvement is the integration of the system with existing hospital information systems (HIS), electronic health record (EHR) platforms, and national healthcare databases. Such integration would enable real-time data sharing across multiple hospitals and healthcare institutions, improving coordination, accessibility, and continuity of patient care. Additionally, integration with secure digital identity platforms and healthcare authentication services can further enhance user verification and data authenticity.

Another important enhancement is the expansion of AI capabilities within the system. Future versions can incorporate advanced machine learning models for disease prediction, personalized treatment recommendations, and clinical decision support based on historical medical data. More sophisticated natural language processing techniques can also be integrated to improve medical report interpretation, multilingual summaries, and automated clinical documentation for healthcare professionals.

Scalability and deployment improvements can also be considered for large-scale real-world implementation. The system can be migrated to cloud-based infrastructure with containerized deployment using Docker and orchestration tools to support a larger number of users and medical records. Optimized database indexing, caching mechanisms, and load balancing can further enhance system performance and reliability in high-traffic healthcare environments.

The platform can also be extended with a dedicated mobile application to improve accessibility for patients and doctors. Features such as real-time notifications, appointment reminders, medication tracking, and emergency alerts can significantly enhance patient engagement and healthcare responsiveness. Mobile integration would allow users to access medical insights and reports anytime and anywhere.

Additional future enhancements include the incorporation of advanced data visualization dashboards, wearable device integration, and IoT-based health monitoring systems for real-time health tracking. These features can provide continuous health data analysis and proactive healthcare support through intelligent alerts and predictive analytics.

From a security perspective, future versions can implement multi-factor authentication, advanced encryption techniques, and compliance with healthcare data privacy standards such as HIPAA-like frameworks and secure audit logging. These improvements will strengthen data protection, privacy, and regulatory compliance for handling sensitive medical information.

Furthermore, the modular architecture of the system allows easy extension with features such as telemedicine integration, doctor–patient chat systems, automated prescription management, and integration with pharmacy and laboratory services. These enhancements can transform the platform into a comprehensive digital healthcare ecosystem.

Overall, the future enhancements aim to improve scalability, intelligence, accessibility, and interoperability of the system. By integrating advanced AI models, cloud infrastructure, mobile support, and real-time healthcare analytics, the proposed Integrated Patient Care Dashboard can evolve into a robust, intelligent, and large-scale digital healthcare solution suitable for modern smart healthcare environments.

REFERENCES

Journals and Conferences

[1] A. Y. Bischof et al., "A Collection of Components to Design Clinical Dashboards Incorporating Patient-Reported Outcome Measures: Qualitative Study," *Journal of Medical Internet Research*, vol. 26, no. 10, 2024.

[2] D. Keszthelyi et al., "Patient Information Summarization in Clinical Settings: A Scoping Review," *JMIR Medical Informatics*, vol. 11, 2023.

[3] E. Coiera et al., "Clinical and economic impact of digital dashboards on hospital inpatient care: A systematic review," *Journal of the American Medical Informatics Association Open*, vol. 8, no. 4, 2025.

[4] C. Lee, K. A. Vogt, and S. Kumar, "Prospects for AI clinical summarization to reduce the burden of patient chart review," *Frontiers in Digital Health*, vol. 6, 2024.

[5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[6] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, 1988. (TF-IDF foundation)

Websites

[7] React Documentation, "React – A JavaScript Library for Building User Interfaces," Available: https://react.dev

[8] FastAPI Documentation, "FastAPI – Modern, Fast Web Framework for Python," Available: https://fastapi.tiangolo.com

[9] Supabase Documentation, "Supabase – Open Source Backend as a Service," Available: https://supabase.com/docs

[10] MongoDB Inc., "MongoDB Documentation," Available: https://www.mongodb.com/docs

[11] Docker Inc., "Docker Documentation," Available: https://docs.docker.com

[12] Python Software Foundation, "Python Machine Learning Libraries Documentation," Available: https://scikit-learn.org

CO-PO-PSO Mapping

Project Title: Automated Healthcare Diagnostic System using AutoML

Course Outcomes (COs)

- CO1: Understand the fundamentals of blockchain and its applications in healthcare systems.
- CO2: Design and develop a full-stack application integrated with blockchain technology.
- CO3: Implement secure data storage and verification mechanisms using cryptographic techniques.
- CO4: Analyse system performance, security, and scalability for real-world applications.

COs	Program Outcomes (POs)												PSOs		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO1	PSO2	PSO3
CO 1	3	2	1	1	2	-	-	-	-	-	-	2	3	2	1
CO 2	2	3	3	2	3	-	-	-	1	1	-	2	3	3	2
CO 3	2	2	3	3	2	-	-	-	1	-	-	2	2	3	2
CO 4	3	2	2	3	2	-	-	-	-	-	-	2	2	2	3

Scale: 3 - High 2 - Medium 1 - Low - Not Applicable

PO / PSO	Level	Justification
PO1	3	Engineering knowledge of blockchain fundamentals, distributed ledger systems, and software development was applied to understand and explain the healthcare application framework.
PO2	2	The healthcare data management problem was analyzed using research literature on blockchain technology and existing limitations in centralized systems.
PO3	1	The project demonstrates an overview of system design through integration of blockchain, mobile application, and backend API components.
PO4	1	Basic investigation of blockchain consensus mechanisms and cryptographic principles was conducted to understand the foundational concepts.

PO5	2	Modern tools such as ReactJS, FastAPI, Supabase, MongoDB, and blockchain frameworks were studied and applied during the understanding phase.
PO12	2	Continuous learning of blockchain principles and emerging healthcare technologies was required to comprehend the full scope of the project.
PSO1	3	Programming knowledge in Python and JavaScript was applied to implement and understand system components related to blockchain and healthcare data.
PSO2	2	Artificial intelligence and cloud technologies relevant to the healthcare dashboard were studied and understood through this outcome.
PSO3	1	Professional awareness of responsible healthcare technology development was established through understanding blockchain applications in sensitive domains.

PO / PSO	**Level**	**Justification**
PO1	2	Knowledge of software engineering, full-stack development, and blockchain integration was applied to design and build the patient care dashboard.
PO2	3	The healthcare management problem was identified and systematically analyzed, leading to the design of a centralized intelligent dashboard addressing communication gaps and data fragmentation.
PO3	3	A complete system including ReactJS frontend, FastAPI backend, hybrid databases (Supabase and MongoDB), blockchain layer, and AI analytical modules was designed and developed.
PO4	2	Investigation of system architecture decisions, API design patterns, and blockchain integration strategies was conducted during development.
PO5	3	Modern tools including ReactJS, FastAPI, Docker, Supabase, MongoDB, and blockchain frameworks were effectively applied in building the full-stack application.

PO9	1	The development process involved coordination between frontend, backend, AI, and blockchain components, requiring basic individual and team project management skills.
PO10	1	Technical design decisions and development progress were communicated through structured documentation and system diagrams.
PO12	2	Learning of full-stack development practices, blockchain integration patterns, and modern cloud deployment techniques was essential for developing the application.
PSO1	3	Programming languages Python and JavaScript were extensively used for developing system components including API services, frontend interfaces, and blockchain interactions.
PSO2	3	Artificial intelligence and cloud computing technologies were integrated to build a deployable, intelligent patient care dashboard.
PSO3	2	Professional software engineering practices including modular design, documentation, and structured development were demonstrated throughout system development.

PO / PSO	**Level**	**Justification**
PO1	2	Mathematical and algorithmic knowledge of cryptographic techniques such as hashing, digital signatures, and smart contracts was applied for secure data storage.
PO2	2	Security vulnerabilities and data integrity challenges in existing healthcare systems were analyzed to design appropriate cryptographic solutions.
PO3	3	Secure data storage mechanisms, role-based access control, blockchain verification layers, and encrypted data handling were implemented as part of the system architecture.
PO4	3	Experimental investigation of cryptographic algorithm performance, smart contract execution, and data verification accuracy was conducted.

PO5	2	Cryptographic libraries, blockchain development tools, and secure API communication frameworks were used to implement data security mechanisms.
PO9	1	Collaborative efforts in implementing and integrating secure data components across the full-stack system were demonstrated.
PO12	2	Continuous learning of cryptographic standards, blockchain security protocols, and emerging data privacy regulations was required.
PSO1	2	Programming skills were applied to implement cryptographic algorithms and blockchain smart contracts for data verification.
PSO2	3	Advanced AI and cloud technologies were integrated with blockchain-based security mechanisms to ensure end-to-end data protection.
PSO3	2	Responsible and ethical implementation of data security practices reflecting professional standards in healthcare software development was demonstrated.

PO / PSO	**Level**	**Justification**
PO1	3	Engineering knowledge of performance benchmarking, system scalability, and security analysis was applied to evaluate the real-world readiness of the application.
PO2	2	System behaviour, API response times, data processing workflows, and blockchain transaction throughput were analyzed to identify performance bottlenecks.
PO3	2	System evaluation involved assessing the architectural components for scalability, latency, and fault tolerance under various operational scenarios.
PO4	3	Comprehensive testing, performance profiling, security auditing, and system validation were conducted under multiple real-world operational conditions.
PO5	2	Testing tools, API profiling frameworks, and security evaluation utilities were used to assess system performance and identify vulnerabilities.

PO12	2	Continuous learning of system optimization techniques, performance benchmarking methodologies, and modern testing frameworks was required for effective analysis.
PSO1	2	Programming and scripting skills were used to design and execute test cases, analyse results, and implement performance improvements.
PSO2	2	AI-assisted analytical modules and cloud-based deployment infrastructure were evaluated for performance, reliability, and security compliance.
PSO3	3	Systematic analysis, critical evaluation, and structured reporting of system performance reflect professional engineering problem-solving and analytical competencies.

Sustainable Development Goals (SDGs)

SDG No.	SDG Title
SDG 3	Good Health and Well-Being
SDG 9	Industry, Innovation and Infrastructure
SDG 4	Quality Education

CO–SDG Mapping with Justification

Table 12.CO	SDG No.	SDG Title
CO1	4	Quality Education
CO2	3, 9	Good Health and Well-Being / Industry, Innovation and Infrastructure
CO3	9	Industry, Innovation and Infrastructure
CO4	3, 9	Good Health and Well-Being / Industry, Innovation and Infrastructure

CO–SDG Justification

CO1 – SDG 4 (Quality Education)

- Demonstrates technical knowledge of blockchain systems and software development through project documentation and presentation.
- Supports engineering education by applying theoretical knowledge of distributed systems and cryptographic principles to practical healthcare application development.

CO2 – SDG 3 & SDG 9 (Good Health and Well-Being / Industry, Innovation and Infrastructure)

- The developed full-stack patient care dashboard promotes efficient and accessible healthcare data management, contributing to improved patient well-being.

- The system uses modern technologies such as blockchain, artificial intelligence, and cloud deployment to support digital healthcare innovation and infrastructure advancement.

CO3 – SDG 9 (Industry, Innovation and Infrastructure)

- The project implements innovative cryptographic and blockchain-based mechanisms for secure data storage, contributing to next-generation healthcare infrastructure.
- Role-based access control and smart contract verification advance the reliability and integrity of digital health systems.

CO4 – SDG 3 & SDG 9 (Good Health and Well-Being / Industry, Innovation and Infrastructure)

- Performance and security analysis ensures the developed digital healthcare system is reliable and safe for real-world patient data management.